

Report delivered to the target repo: [shende-shweta/FSDKC PR #7](#) → `docs/discovery/07-technical-debt.md` on branch `discovery/technical-debt-20260723T184156_wzru8a` . The orchestration UI will convert it to `docs/discovery/07-technical-debt.pdf` .

Below are the required sections, verbatim from the saved report.

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

Moderate

Driven by no enforced lint/static-analysis (D5), a weak CI gate with missing composer.lock and no dev-api job (D1/D3), and hand-managed schema with missing indexes and shared MongoDB collections (D4).

EXECUTIVE SUMMARY

Klearcom is a well-structured polyglot monolith with genuinely strong agentic breadcrumbs — five `AGENTS.md` files, Kiro MCP scaffolding, and an already-enumerated defect backlog (`docs/CODEBASE_AUDIT_ISSUES.md`) that reads like a ready-made agent work queue. The foundations are decent: a top-level `.gitignore`, a two-job GitHub Actions pipeline, MariaDB foreign keys with cascade deletes, `.env.example` files, and a complete Docker Compose stack. However, the repository is **not yet ready to trust agent-authored output end-to-end**. The three most severe gaps are: (1) **no code-style or static-analysis enforcement anywhere** — no ESLint/Prettier/Pint config exists, `phpstan` is declared but never wired or run, and the documented "no `extract()`" convention is already violated in committed code; (2) **a weak CI gate** — the backend job installs without a committed `composer.lock`, the frontend job only runs `build` (no lint, no tests), and the `dev-api` Node service has no CI job at all; and (3) **schema managed by hand-written `init.sql` with no migration framework, no secondary indexes on foreign-key columns, and shared "flat" MongoDB collections** that block clean service extraction. Compounding this, the Laravel backend and the Node `dev-api` maintain **two divergent implementations** of the same business logic. Honest verdict: agentic-readiness is **Moderate** — the work is enumerable and isolated, but the CI verification gate an agent's changes would land against cannot yet be trusted.

Readiness Benchmark Ratings

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	<code>.gitignore</code> + CI + JS lock files present, but no <code>composer.lock</code> , no branch protection / CODEOWNERS , committed <code>.tsbuildinfo</code> build artifacts	MODERATE
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	13/14 relevant deps wired; <code>phpstan</code>	MODERATE

					declared (dev) but no config and never run	
D3	AI Tool / Agentic Readiness	ready	partial	not ready	Excellent discovery signals (5× <code>AGENTS.md</code> , Kiro, enumerated backlog) but CI gate cannot verify agent output (no lint, <code>dev-api</code> untested)	MODERATE
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	FKs + cascade present, but no indexes on FK columns, no migration framework (manual <code>init.sql</code>), shared flat Mongo collections	MODERATE
D5	Development Environment	reproducible	partial	manual / fragile	<code>.env.example</code> + Docker + OS-portable, but zero code-style/static-analysis enforcement ; conventions documented yet violated	MODERATE
D6	Divergent dual implementation (<i>additional</i>)	single source of truth	some duplication across stacks	fully forked parallel backends	Laravel backend and Node <code>dev-api</code> re-implement the same KPI/tree/reachability logic (audit §4) — drift risk	MODERATE

No additional readiness gaps beyond D6 were observed.

7.6 Prerequisites for Agentic Harness & Marketplace Readiness

Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	Strong. <code>docs/CODEBASE_AUDIT_ISSUES.md</code> enumerates 14 issue clusters with exact <code>file:line</code> refs and expected fixes.	Backlog is a static doc, not a machine-readable/issue-tracked queue. Convert to labelled GitHub issues or a <code>tasks.json</code> for automation.
Isolated, verifiable units of work	Partial. Module folders, a service layer to extract into, and repeated patterns make tasks isolable.	Isolation is undermined by duplicated logic across the two backends (§D6) — a fix in one may need mirroring in the other.
CI gate to accept agent-authored output	Weak. CI runs backend phpunit + frontend build only.	No lint/static-analysis; <code>dev-api</code> has no CI job; missing <code>composer.lock</code> ; thin coverage of the exact logic under refactor. Agent output cannot be auto-trusted.
Repo hygiene for automation (clean checkout, no secrets)	Moderate. <code>.gitignore</code> present, no secrets committed (only local dev creds in <code>.env.example</code> /compose).	Committed <code>*.tsbuildinfo</code> artifacts and missing <code>composer.lock</code> make "clean, reproducible checkout" not fully guaranteed.

Marketplace packaging readiness	Early. Docker Compose gives a runnable stack; module boundaries + <code>AGENTS.md</code> exist.	No versioned/publishable package boundaries, no production frontend image, no release/versioning workflow.
---------------------------------	--	--

7.8 Actions Required

Gap	Action	Rating	Priority
No code-style / static-analysis enforcement (D5)	Add ESLint+Prettier (JS/TS) and Pint+PHPStan (PHP) with configs; wire into CI and a pre-commit hook; enforce the documented "no <code>extract()</code> " rule	MODERATE	CRITICAL
Weak CI gate for agent output (D3)	Add a <code>dev-api</code> CI job (install + smoke/test), add lint steps to all jobs, and add tests for reachability/KPI/realtime logic so CI can fail on bad changes	MODERATE	HIGH
Missing <code>composer.lock</code> (D1)	Commit <code>backend/composer.lock</code> and switch CI to install from the lock for reproducible backend builds	MODERATE	HIGH
No migration framework; manual <code>init.sql</code> (D4)	Introduce Laravel migrations (or a versioned SQL migration tool) as the schema source of truth with rollback support	MODERATE	HIGH
No indexes on FK / lookup columns (D4)	Add indexes on <code>discovery_job_id</code> , <code>parent_id</code> , <code>connect_monitor_id</code> , and <code>(module, reference_id)</code> collections	MODERATE	MEDIUM
<code>phpstan</code> declared but never wired (D2)	Add <code>phpstan.neon</code> and run it in CI, or remove the unused dev dependency	MODERATE	MEDIUM
No branch protection / CODEOWNERS / PR template (D1)	Enable required status checks + review on <code>main</code> ; add <code>CODEOWNERS</code> and a PR template	MODERATE	MEDIUM
Committed <code>*.tsbuildinfo</code> build artifacts (D1)	Add <code>*.tsbuildinfo</code> to <code>.gitignore</code> and untrack the two committed files	MODERATE	LOW
Divergent dual backend implementations (D6)	Establish a single source of truth (extract shared logic to services / a shared package) or clearly demote <code>dev-api</code> to a mock so KPI/tree/reachability logic cannot drift	MODERATE	MEDIUM
Shared flat MongoDB collections (D4)	Plan per-domain collections/DBs before any Discovery/Connect service split	MODERATE	LOW
Frontend container is dev-only (D5)	Add a production build/serve stage to <code>frontend/Dockerfile</code> before deployment	MODERATE	LOW